

REST API Reference

Base URL: `http://<host>:<port>/api` (default `http://localhost:3000/api`)

All responses are JSON. All endpoints except `/health` , `/schema` and `/auth/login` require a **Bearer token** obtained from `/auth/login` :

Authorization: Bearer <token>

For file downloads opened directly in the browser you may instead pass the token as a query parameter: `?token=<token>`.

Authentication

POST /auth/login

Authenticate and receive a JWT.

Request:

```
{ "username": "alice", "password": "pw" }
```

Response 200 :

```
{
  "token": "eyJhbGciOi...",
  "user": { "id": "...", "username": "alice", "role": "investigator", "public_key": "..."}
}
```

Errors: 401 invalid credentials.

GET /auth/me

Return the authenticated user's profile.

Health & schema

GET /health

No auth. Returns chain height, user count and NTP health.

```
{ "status": "ok", "height": 12, "users": 3,
  "time": { "ntp_enabled": true, "reachable": true, "authority": "pool.ntp.org", "offset seconds": 0.01 } }
```

GET /schema

No auth. Returns the JSON Schema used to validate evidence metadata.

Evidence

POST /evidence (permission: evidence:create)

Submit one or more evidence files under a single evidence block.

Content-Type: multipart/form-data

Field	Type	Notes
metadata	string (JSON)	Must validate against the evidence schema
files	file(s)	One or more; repeat the field for multiple files

metadata example:

```
{
  "case_id": "CASE-2026-001",
  "operation_id": "OP-ALPHA",
  "title": "Suspect tweet",
  "source_type": "social_media",
  "platform": "twitter/x",
  "author_handle": "@suspect",
  "source_url": "https://x.com/...",
  "classification": "UNCLASSIFIED",
  "tags": ["recruitment"]
}
```

Response 201 : the sealed block (including block_id , merkle_root , block_hash , signature , and per-file items[] with file_hash).

Errors: 400 metadata validation failed (with details[]), 403 no access to this case .

POST /evidence/<parent_block_id>/derived (permission: evidence:create)

Add a derived artifact (OCR, translation, thumbnail...) linked to a parent block.

Content-Type: multipart/form-data

Field	Type	Notes
metadata	string (JSON)	schema-validated
derivation_type	string	e.g. ocr , translation , thumbnail
parent_file_hash	string	hash of the source file in the parent block
tool	string	optional tool name (e.g. tesseract)
files	file(s)	the derived output file(s)

GET /evidence (permission: evidence:read)

List/search evidence and derived blocks. Query params (all optional):

case_id , q (free text), source_type , tag . Results are filtered to cases the user can access.

GET /evidence/<block_id> (permission: evidence:read)

Return a single block enriched with derived[] children. **Side effect:** logs a view access block.

GET /evidence/<block_id>/download/<file_hash> (permission: evidence:export)

Download an original file. **Side effect:** logs an export access block.

Chain & verification

GET /chain (permission: evidence:read)

Return the full chain: { "height": N, "blocks": [...] } .

GET /chain/verify (permission: chain:verify)

Run full integrity verification.

```
{ "valid": true, "height": 12, "checked": 12, "errors": [] }
```

Checks performed per block: hash integrity, Ed25519 signature, index/prev-hash linkage, Merkle root consistency, and (if registered) collector public-key identity match.

GET /verify/file/<file_hash> (permission: chain:verify)

Re-hash the stored file and compare to the on-chain hash.

```
{ "found": true, "intact": true, "recorded_hash": "...", "actual_hash": "..." }
```

Audit log

GET /audit (permission: audit:read)

Return access blocks. Optional target_block_id filter. Each record contains action (view / export / transfer / download / search), actor_id , target_block_id , details and the trusted timestamp .

User management (permission: user:manage , admin only)

GET /users

List all users (public view, no secrets).

POST /users

Create a user. Body:

```
{ "username": "bob", "password": "pw", "role": "investigator",  
  "full_name": "Bob", "cases": ["CASE-1"] }
```

A fresh Ed25519 keypair is generated automatically. Response 201 .

PATCH /users/<user_id>

Update a user. Any subset of:

```
{ "role": "viewer", "active": false, "password": "new",  
  "assign_case": "CASE-2", "revoke_case": "CASE-1", "rotate_keys": true }
```

Block structure

Every block has this shape (signable fields are hashed & signed):

```
{
  "index": 1,
  "block_id": "hex-uuid",
  "block_type": "evidence",           // genesis | evidence | derived | access
  "prev_hash": "sha256...",
  "timestamp": { "iso": "...", "unix": 1.0, "authority": "pool.ntp.org",
                 "source": "ntp", "ntp_offset": 0.01 },
  "collector_id": "user-id",
  "collector_pubkey": "ed25519-hex",
  "merkle_root": "sha256... | null",
  "payload": { ... },                // shape depends on block_type
  "block_hash": "sha256...",         // SHA-256 of the signable content
  "signature": "ed25519-hex"        // signature of block_hash by collector
}
```

NEW HybridChain-OSINT OS v2.0 Features

Collection Tasks (Intelligence Gathering)

Investigators publicly post evidence collection requirements that the crowdsourced community helps fulfill. This enables distributed intelligence gathering at scale.

POST /api/tasks

Create a new collection task (investigators only).

Requires: **investigator** or **admin** role.

Request:

```
{
  "title": "Screenshots of phishing campaign targeting financial sector",
  "description":
    "Need evidence of emails claiming to be from Bank XYZ asking users to verify accounts.
    Include full headers if possible.",
  "evidence_types": ["screenshot", "document"],
  "case_id": "CASE-001",
  "priority": "high",
  "quantity_needed": 10,
  "deadline": 1735689600.0,
  "metadata_requirements": {
    "source": "email",
    "must_include": ["sender_address", "received_date"]
  }
}
```

Response 201 :

```
{
  "task_id": "task-uuid-123",
  "investigator_id": "investigator-xyz",
  "title": "Screenshots of phishing campaign targeting financial sector",
  "status": "open",
  "quantity_needed": 10,
  "quantity_fulfilled": 0,
  "progress_percentage": 0,
  "created_at": 1234567890.0,
  "submissions": [],
  "accepted_submissions": []
}
```

GET /api/tasks

List collection tasks with optional filters.

Query parameters:

- status : open | in_progress | partially_fulfilled | fulfilled | closed
- case_id : filter by case
- priority : low | medium | high | urgent
- investigator_id : filter by investigator
- include_expired : true | false (default: false)

Response 200 :

```
{
  "tasks": [
    {
      "task_id": "task-uuid-123",
      "investigator_id": "investigator-xyz",
      "title": "Screenshots of phishing campaign targeting financial sector",
      "description": "Need evidence of emails...",
      "evidence_types": ["screenshot", "document"],
      "case_id": "CASE-001",
      "priority": "high",
      "quantity_needed": 10,
      "quantity_fulfilled": 3,
      "progress_percentage": 30.0,
      "deadline": 1735689600.0,
      "status": "in_progress",
      "submissions": ["sub-1", "sub-2", "sub-3", "sub-4"],
      "accepted_submissions": ["sub-1", "sub-2", "sub-3"],
      "created_at": 1234567890.0,
      "updated_at": 1234567920.0,
      "is_expired": false
    }
  ],
  "count": 1
}
```

GET /api/tasks/<task_id>

Get details of a specific collection task.

Response 200 :

```
{
  "task_id": "task-uuid-123",
  "investigator_id": "investigator-xyz",
  "title": "Screenshots of phishing campaign targeting financial sector",
  "description": "Need evidence of emails...",
  "evidence_types": ["screenshot", "document"],
  "case_id": "CASE-001",
  "priority": "high",
  "quantity_needed": 10,
  "quantity_fulfilled": 3,
  "progress_percentage": 30.0,
  "deadline": 1735689600.0,
  "metadata_requirements": {
    "source": "email",
    "must_include": ["sender_address", "received_date"]
  },
  "status": "in_progress",
  "submissions": ["sub-1", "sub-2", "sub-3", "sub-4"],
  "accepted_submissions": ["sub-1", "sub-2", "sub-3"],
  "created_at": 1234567890.0,
  "updated_at": 1234567920.0,
  "is_expired": false
}
```

PATCH /api/tasks/<task_id>/close

Close a collection task (stop accepting new submissions).

Requires: **investigator** or **admin** role (task owner only).

Response 200 :

```
{
  "task_id": "task-uuid-123",
  "status": "closed",
  "message": "Task closed successfully"
}
```

GET /api/tasks/statistics

Get overall collection task statistics.

Response 200 :

```
{
  "total_tasks": 45,
  "open_tasks": 12,
  "fulfilled_tasks": 20,
  "in_progress_tasks": 10,
  "closed_tasks": 3,
  "total_submissions": 234,
  "total_accepted_submissions": 156,
  "acceptance_rate": 66.7
}
```

Public Blockchain (Verification Layer)

The public blockchain contains only verification metadata without sensitive evidence data. It enables community participation in validation while maintaining confidentiality.

GET /api/public-chain

Get the entire public verification blockchain.

Response 200 :

```
{
  "chain": [
    {
      "index": 0,
      "timestamp": 1234567890.0,
      "private_block_hash": "abc123...",
      "public_metadata": {
        "case_id": "CASE-001",
        "source": "Twitter",
        "classification": "public",
        "evidence_count": 3
      },
      "verification_status": "verified",
      "verifier_signatures": [
        {
          "verifier_id": "user-xyz",
          "verifier_pubkey": "abc...",
          "signature": "def...",
          "decision": "verified",
          "timestamp": 1234567891.0
        }
      ],
      "prev_hash": "",
      "nonce": 0,
      "hash": "public_block_hash_123..."
    }
  ],
  "height": 42
}
```

GET /api/public-chain/verify

Verify the integrity of the public blockchain.

Response 200 :

```
{
  "valid": true,
  "errors": [],
  "checks": {
    "hash_integrity": "pass",
    "chain_linkage": "pass",
    "cross_chain_verification": "pass"
  }
}
```

Response 400 (if invalid):

```
{
  "valid": false,
  "errors": [
    "Block 5: Hash mismatch",
    "Block 7: Invalid prev_hash linkage"
  ]
}
```

GET /api/public-chain/pending

List all evidence blocks pending community verification.

Response 200 :

```
{
  "pending": [
    {
      "index": 10,
      "private_block_hash": "xyz789...",
      "public_metadata": { "case_id": "CASE-002", "source": "Facebook" },
      "verification_status": "pending",
      "verifier_signatures": [],
      "timestamp": 1234567900.0
    }
  ],
  "count": 1
}
```

GET /api/cross-verify/<private_block_id>

Verify that a private block correctly links to its public block.

Response 200 :

```
{
  "private_block_id": "block-abc123",
  "public_block_index": 5,
  "linked": true,
  "private_hash": "abc123...",
  "public_private_hash": "abc123...",
  "verification_status": "verified"
}
```

Crowdsourcing & Community Verification

Community members can submit evidence and participate in verification workflows with a reputation-based system.

POST /api/crowdsource/register

Register as a community member. No authentication required for initial registration.

Request:

```
{
  "username": "analyst_01",
  "public_key": "ed25519_hex_pubkey...",
  "email": "analyst@example.com"
}
```

Response 201 :

```
{
  "member_id": "member-uuid-123",
  "username": "analyst_01",
  "role": "public_contributor",
  "reputation": 0,
  "join_date": 1234567890.0
}
```

POST /api/crowdsourcing/submit

Submit evidence for community verification.

Can be linked to a collection task to fulfill an investigator's evidence requirement.

Requires authentication (community member token).

Request (multipart/form-data):

```
submitter_id: member-uuid-123
case_id: CASE-001 (optional)
task_id: task-uuid-123 (optional - links to collection task)
metadata: { "source": "Reddit", "description": "...", ... }
files[]: file1.jpg, file2.png
```

Response 201 :

```
{
  "submission_id": "submission-uuid-456",
  "task_id": "task-uuid-123",
  "status": "pending",
  "timestamp": 1234567890.0,
  "submitter_reputation_change": +5,
  "task_progress": "3/10 (30%)"
}
```

GET /api/crowdsourcing/submissions

List community submissions, optionally filtered.

Query parameters:

- status : pending | accepted | rejected
- case_id : filter by case
- submitter_id : filter by submitter

Response 200 :

```
{
  "submissions": [
    {
      "submission_id": "submission-uuid-456",
      "submitter_id": "member-uuid-123",
      "evidence_data": { "source": "Reddit", "description": "..."},
      "timestamp": 1234567890.0,
      "status": "pending",
      "votes": [
        {
          "voter_id": "member-uuid-789",
          "vote": "authentic",
          "weight": 1.5,
          "comment": "Metadata matches known patterns",
          "timestamp": 1234567900.0
        }
      ],
      "consensus_reached_at": null
    }
  ],
  "count": 1
}
```

POST /api/crowdsource/submissions/<submission_id>/vote

Vote on a community submission.

Requires: **verified_analyst** role or higher.

Request:

```
{
  "vote": "authentic", // authentic | suspicious | fabricated | needs_more_info
  "comment": "Cross-referenced with known account activity patterns"
}
```

Response 200 :

```
{
  "vote_recorded": true,
  "vote_weight": 1.5,
  "new_status": "accepted", // or "pending" if consensus not reached
  "consensus_threshold": 0.6,
  "current_authentic_ratio": 0.75,
  "submitter_reputation_change": +20,
  "voter_reputation_change": +10
}
```

Errors:

- 403 : Insufficient role (must be verified_analyst or higher)
- 404 : Submission not found
- 400 : Already voted on this submission

GET /api/crowdsource/leaderboard

Get top contributors by reputation.

Query parameters:

- `limit` : Number of members to return (default 10)

Response 200 :

```
{
  "leaderboard": [
    {
      "member_id": "member-uuid-123",
      "username": "analyst_01",
      "role": "senior_analyst",
      "reputation": 1250,
      "verifications_count": 45,
      "submissions_count": 12,
      "verification_accuracy": 0.92
    },
    {
      "member_id": "member-uuid-789",
      "username": "investigator_02",
      "role": "verified_analyst",
      "reputation": 850,
      "verifications_count": 30,
      "submissions_count": 8,
      "verification_accuracy": 0.87
    }
  ]
}
```

GET /api/crowdsource/members/<member_id>

Get community member profile and statistics.

Response 200 :

```
{
  "member_id": "member-uuid-123",
  "username": "analyst_01",
  "role": "verified_analyst",
  "reputation": 850,
  "join_date": 1234567890.0,
  "verifications_count": 30,
  "submissions_count": 8,
  "verification_accuracy": 0.87,
  "public_key": "ed25519_hex...",
  "recent_activity": [
    {
      "action": "verify_correct",
      "timestamp": 1234567950.0,
      "reputation_change": +10
    },
    {
      "action": "evidence_verified",
      "timestamp": 1234567920.0,
      "reputation_change": +20
    }
  ]
}
```

PATCH /api/crowdsourcing/members/<member_id>/promote

Promote a member to a higher role (admin only).

Requires: **admin** role on private chain.

Request:

```
{
  "new_role": "senior_analyst" // verified_analyst | senior_analyst | moderator
}
```

Response 200 :

```
{
  "member_id": "member-uuid-123",
  "username": "analyst_01",
  "old_role": "verified_analyst",
  "new_role": "senior_analyst",
  "new_vote_weight": 2.0
}
```

Community Roles & Permissions

Role	Submit Evidence	Vote/Verify	Base Vote Weight	Min. Reputation
public_contributor	✓	✗	0.0	0
verified_analyst	✓	✓	1.0	50+
senior_analyst	✓	✓	2.0	200+
moderator	✓	✓	3.0	500+

Vote Weight Calculation:

```
final_weight = base_weight × reputation_multiplier × accuracy_multiplier
where:
  reputation_multiplier = min(1.0 + (reputation / 1000), 2.0)
  accuracy_multiplier = 0.5 + (verification_accuracy × 0.5)
```

Reputation Actions & Points

Action	Points	Description
Submit Evidence	+5	For each evidence submission
Verify Correct	+10	When your vote aligns with consensus
Verify Incorrect	-15	When your vote contradicts consensus
Evidence Verified	+20	When your submission is accepted
Evidence Rejected	-10	When your submission is rejected
Help Solve Case	+100	Special recognition for case resolution

Consensus Mechanism

Threshold: 60% weighted vote (configurable)

Example:

- Submission receives 5 votes
- 3 votes "authentic" (weights: 1.0, 1.5, 2.0) = 4.5 total
- 2 votes "fabricated" (weights: 1.0, 1.0) = 2.0 total
- Total weight: 6.5
- Authentic ratio: $4.5 / 6.5 = 0.69$ (69%)
- **Result:** ACCEPTED (exceeds 60% threshold)

Status changes:

- pending → accepted when `authentic_ratio` ≥ 0.6
- pending → rejected when `fabricated_ratio` ≥ 0.6
- Remains pending otherwise

Error Responses

All endpoints may return these standard errors:

401 Unauthorized:

```
{ "error": "No token provided" }
{ "error": "Invalid or expired token" }
```

403 Forbidden:

```
{ "error": "Insufficient permissions" }
{ "error": "Role 'viewer' cannot perform this action" }
```

404 Not Found:

```
{ "error": "Resource not found" }
```

400 Bad Request:

```
{ "error": "Validation failed", "details": "..." }
```

500 Internal Server Error:

```
{ "error": "Internal server error", "message": "..." }
```

Integration Examples

Cross-Chain Workflow

```
# 1. Investigator submits sensitive evidence to private chain
curl -X POST http://localhost:3000/api/evidence \
  -H "Authorization: Bearer <investigator_token>" \
  -F "case_id=CASE-001" \
  -F "source=Twitter" \
  -F "classification=internal" \
  -F "files[]=@screenshot.png"

# Returns: { "block_id": "block-abc123", ... }

# 2. System automatically creates public verification block
# (only metadata, no sensitive data)

# 3. Community analyst votes on authenticity
curl -X POST http://localhost:3000/api/crowdsourcing/public-blocks/5/vote \
  -H "Authorization: Bearer <community_token>" \
  -d '{"vote": "authentic", "comment": "Metadata verified"}'

# 4. Cross-verify linkage
curl http://localhost:3000/api/cross-verify/block-abc123

# Returns: { "linked": true, "verification_status": "verified" }
```

Crowdsourced Investigation with Collection Tasks

```
# 1. Investigator posts collection requirement
curl -X POST http://localhost:3000/api/tasks \
-H "Authorization: Bearer <investigator_token>" \
-d '{
  "title": "Screenshots of phishing emails",
  "description": "Need examples of recent phishing campaign",
  "evidence_types": ["screenshot"],
  "priority": "high",
  "quantity_needed": 10
}'

# Returns: { "task_id": "task-123", "status": "open", ... }

# 2. Community views open tasks
curl http://localhost:3000/api/tasks?status=open

# 3. Public contributor submits evidence for the task
curl -X POST http://localhost:3000/api/crowdsource/submit \
-H "Authorization: Bearer <contributor_token>" \
-F "task_id=task-123" \
-F "metadata={\"source\": \"Email\", ...}" \
-F "files[]=@phishing_screenshot.jpg"

# Returns: { "submission_id": "sub-456", "task_progress": "1/10 (10%)", ... }

# 4. Verified analysts vote on authenticity
curl -X POST http://localhost:3000/api/crowdsource/submissions/sub-456/vote \
-H "Authorization: Bearer <analyst_token>" \
-d '{"vote": "authentic", "comment": "Verified against known phishing patterns"}'

# 5. Once consensus reached, system accepts submission
# Task automatically updates: "task_progress": "2/10 (20%)"

# 6. After validation, admin imports to private chain
curl -X POST http://localhost:3000/api/evidence/import-from-crowdsource \
-H "Authorization: Bearer <admin_token>" \
-d '{"submission_id": "sub-456"}'

# 7. Check task progress
curl http://localhost:3000/api/tasks/task-123
# Returns: { "quantity_fulfilled": 2, "progress_percentage": 20.0, ... }
```

Traditional Crowdsourced Investigation (No Task)

```
# Community can also submit evidence not linked to specific tasks

# 1. Public contributor submits evidence
curl -X POST http://localhost:3000/api/crowdsourcing/submit \
  -H "Authorization: Bearer <contributor_token>" \
  -F "case_id=CASE-002" \
  -F "metadata={\"source\": \"Reddit\",...}" \
  -F "files[]=@evidence.jpg"

# 2. Verified analysts vote
curl -X POST http://localhost:3000/api/crowdsourcing/submissions/<id>/vote \
  -H "Authorization: Bearer <analyst_token>" \
  -d '{"vote": "authentic", "comment": "..."}'

# 3. Once consensus reached, admin adds to private chain
curl -X POST http://localhost:3000/api/evidence/import-from-crowdsourcing \
  -H "Authorization: Bearer <admin_token>" \
  -d '{"submission_id": "<id>"}'
```

HybridChain-OSINT OS v2.0 - Where private investigations meet public verification.